Figure 2: Visualization of our backtracking method, which is used to feed mistake location information to the model for correction. t refers to the temperature used during generation.

	With **mistake** location		With **random** location		Avg. num.
Task	**Δ accuracy ✓**	**Δaccuracy✗**	**Δ accuracy ✓**	**Δaccuracy✗**	of steps
Word sorting	-11.11	+23.53	-15.56	+11.76	11.7
Tracking shuffled objects	-6.67	+43.92	-6.67	+20.39	5.4
Logical deduction	-11.43	+36.86	-13.33	+21.57	8.3
Multistep arithmetic	-0.00	+18.04	-8.89	+10.59	5.0
Dyck languages	-6.82	+18.06	-15.91	+5.16	24.5

Table 6: Absolute differences in accuracy$_{ans}$ before and after backtracking. "With mistake location" indicates that backtracking was done using oracle mistake locations from the dataset; "With random location" indicates that backtracking was done based on randomly selected locations. **Δaccuracy✓** refers to differences in accuracy$_{ans}$ on the set of traces whose *original* answer was correct$_{ans}$; **Δaccuracy✗** for traces whose original answer was incorrect$_{ans}$. The average number of steps in a trace is shown to demonstrate the likelihood of randomly selecting the correct mistake location in the random baseline condition.

Our results show that, with mistake location information available, LLMs can correct their own outputs and improve overall downstream performance. This suggests that the main bottleneck in self-correction methods is the identification of mistakes, rather than the correction process. This bottleneck can be overcome by using ground truth feedback (as in Reflexion (Shinn et al., 2023) or RCI (Kim et al., 2023)), or by training a classifier (see section 5).

While our numbers do show that our gains are higher than our losses, it should be noted that changes in the overall accuracy depends on the original accuracy achieved on the task. For example, if the original accuracy on the tracking shuffled objects task was 50%, the new accuracy would be 68.6%. On the other hand, if the accuracy was 99%, the new accuracy would drop to 92.8%. As our dataset is highly skewed and only contains 45 correct$_{ans}$ traces per task, we leave to future work a more comprehensive assessment of backtracking, as well as the development of more sophisticated ways to incorporate mistake location information into the self-correction loop.

5 Obtaining mistake location information with a trained classifier

As shown in section 4, if mistake location information is available, LLMs can correct their own CoT traces and boost downstream performance. However, these experimental results are based on oracle labels, which are typically not available in downstream tasks.

One possible solution is to obtain mistake location information from a smaller, trained classifier. If training data is available, one might ask why this approach is preferable to simply fine-tuning the larger, generator model. The reasons are:

- Training a small classifier is far more efficient in terms of computing resources and available data.
- Once the classifier is trained, it can be used with any LLM as the generator and be updated independently. This can be especially helpful with API-based LLMs that cannot be fine-tuned.
- The process of mistake finding is more interpretable than updating the weights of the generator model directly. It clearly pinpoints the location at which an error occurs, which can help the debugging process and allow faster development and iterations of models.

In this section, we seek to answer two questions in the following subsections:

5.1: What mistake-finding accuracy is required for backtracking to be effective?
A trained classifier is unlikely to reach 100% mistake-finding accuracy. If backtracking is only effective when mistake location is 100% accurate, we would not be able to replace oracle labels with a trained classifier.

5.2: Is it possible to improve on results in section 3 without in-domain training data?
Sufficient in-domain training data typically guarantees a performance boost, but can be hard to obtain. We investigate whether mistake-finding in reasoning traces is transferable across tasks. If so, one can use BIG-Bench Mistake or similar datasets to fine-tune a mistake-finding classifier for other tasks.

5.1 Minimum mistake finding accuracy

To explore what level of mistake-finding accuracy is needed, we simulate classifiers at different levels of accuracy and run backtracking for each level. We use accuracy$_{mis}$ to refer to the mistake-finding accuracy classifier, to differentiate from downstream task accuracy$_{ans}$.

For a given classifier at $X\%$ accuracy$_{clf}$, we use the mistake location from BIG-Bench Mistake $X\%$ of the time. For the remaining $(100 - X)\%$, we sample a mistake location randomly. To mimic the behaviour of a typical classifier, mistake locations are sampled to match the distribution found in the dataset. We also ensure that the sampled location does not match the correct location.

Results are shown in Figure 3. We can see that the losses in Δaccuracy$_{\checkmark}$ begins to plateau at 65%.

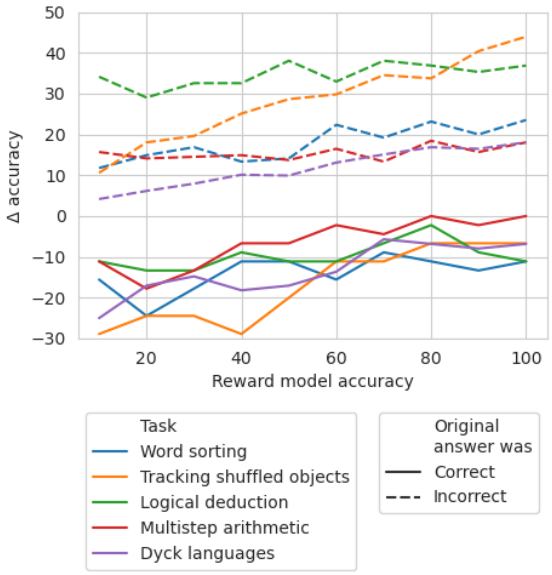

Figure 3: Δaccuracy$_{\checkmark}$ and Δaccuracy$_{\times}$ on each dataset as accuracy$_{clf}$ increases.

In fact, for most tasks, Δaccuracy$_{\checkmark}$ is already larger than Δaccuracy$_{\times}$ at around 60-70% accuracy$_{RM}$. This demonstrates that while higher accuracies produce better results, backtracking is still effective even without gold standard mistake location labels.

5.2 Training a classifier on out-of-domain data

We test whether mistake-finding can benefit from a dedicated classifier trained on out-of-distribution tasks. We fine-tune PaLM 2 Otter, a model much smaller than PaLM 2 Unicorn, with our BIG-Bench Mistake data for 20k steps and choose the checkpoint with the best validation results. For each task, we hold out the in-domain data for evaluation while training the classifier on the other 4 tasks.

We show the relative improvements and losses in Table 7 vs. a 3-shot baseline on PaLM 2 Unicorn (scores from section 3). We see gains for 4 out of 5 of the tasks. Note the classifiers we train are significantly smaller than our inference model, and is trained on out-of-domain data. This suggests that it may be possible to train classifiers to assist in backtracking, and that these classifiers do not have to be large. Further, such a classifier can work on out-of-distribution mistakes.

Despite this, the performance of our classifiers do not meet the threshold required for effective backtracking, as demonstrated in subsection 5.1. We believe more data may be necessary to improve results across the board on all tasks. We leave to future work the collection of this larger dataset and

Held-out task	Trained classifier accuracy$_{mis}$ (Otter)	3-shot prompting accuracy$_{mis}$ (Unicorn)	Difference
Word sorting	**22.33**	11.67	+11.66
Tracking shuffled objects	**37.67**	18.00	+19.67
Logical deduction	6.00	**6.67**	-0.67
Multi-step arithmetic	**26.00**	22.00	+4.00
Dyck languages	**33.57**	10.98	+22.59

Table 7: Absolute difference in mistake finding accuracy between PaLM 2 Unicorn and a small, trained classifier. Bold indicates the best score for each task. Note that PaLM 2 Otter is significantly smaller than PaLM 2 Unicorn, and is trained on out-of-domain data.

a more rigorous investigation of the trade-offs of model size vs. performance of the classifier.

We also leave for future investigation the effect of backtracking iteratively with a classifier: for example, the generator model may make another mistake after backtracking for the first time, which can then be identified and corrected again.

6 Related work

6.1 Datasets

To our knowledge, the only publicly available dataset containing mistake annotations in LLM outputs is PRM800K (Lightman et al., 2023), which is a dataset of solutions to Olympiad-level math questions. Our dataset BIG-Bench Mistake covers a wider range of tasks to explore the reasoning capabilities of LLMs more thoroughly. Additionally, our dataset explores the more general case of prompting API-based LLMs, whereas PRM800K uses a generator LLM that was heavily fine-tuned for math-specific problems.

6.2 Self-correction

The term *self-correction* can be applied to a wide variety of techniques. Pan et al. (2023) present a survey of self-correction methods in recent literature. Broadly, proposed self-correction methods can vary in the following dimensions:

Source of feedback Some techniques rely on external feedback inherent to the task such as code execution errors (Yasunaga and Liang, 2020; Chen et al., 2023a, 2024b), or feedback from humans (e.g. Chen et al., 2024a; Yuan et al., 2024). Some explicitly train a model to produce feedback (e.g. Madaan et al., 2021; Welleck et al., 2023; Bai et al., 2022b; Lee et al., 2023; Paul et al., 2023; Ouyang et al., 2022; Bai et al., 2022a; Ganguli et al., 2023), while others rely on prompting only (e.g. Shinn et al., 2023; Miao et al., 2024). It has been demonstrated that prompting-only setups can work well for stylistic or qualitative improvements (e.g. Madaan et al., 2023; Chen et al., 2023b), but would require external feedback for reasoning tasks (Huang et al., 2023; Shinn et al., 2023; Kim et al., 2023; Madaan et al., 2023).

Time of correction Feedback can be incorporated at various points during the self-correction process. Some methods do so by updating weights during training time (e.g. Ouyang et al., 2022; Bai et al., 2022a; Ganguli et al., 2023); some do so during generation time (e.g. Weng et al., 2023; Dalvi Mishra et al., 2022; Xie et al., 2023); others apply correction to output that has already been generated (e.g. Saunders et al., 2022; Kim et al., 2023; Shinn et al., 2023). Our method falls into the final category (post-hoc correction), as it involves identifying an incorrect step in a complete CoT trace; however, it is also possible to apply a mistake-finding classifier at every step during generation.

7 Conclusion

In this paper, we investigate LLMs' ability to find mistakes and correct outputs. We find that LLMs generally struggle to find mistakes, but, when given mistake location information, are able to correct outputs to boost performance. We therefore hypothesise that mistake finding is an important bottleneck preventing self-corrections strategies from performing well on reasoning tasks.

We show initial evidence that a dedicated classifier for mistake finding can overcome this bottleneck. We train a small, baseline classifier on out-of-domain data and demonstrate improvement over few-shot prompting results. While the classifier does not reach the threshold required for effective backtracking, our results show that it is possible to improve on mistake-finding accuracy using standard machine learning techniques. We leave the development of more sophisticated methods to future work, and release our dataset BIG-Bench Mistake to encourage this direction of research.